Homework 07

```
library(MASS)
```

Question 1: LDA

```
heart <- read.csv("data/saheart.csv")
heart$famhist <- factor(heart$famhist, levels = c("Absent", "Present"))
X <- data.matrix(heart[, names(heart) != "chd"])
y <- heart$chd
n <- nrow(X); p <- ncol(X); K <- 2; classes <- c(0, 1)
cat("X:", dim(X), " y:", length(y), "\n")
```

```
X: 462 9  y: 462
```

a. Priors, class means, and pooled covariance (denominator $n - K$):

```
pi_hat <- sapply(classes, function(k) mean(y == k))
mu <- t(sapply(classes, function(k) colMeans(X[y == k, , drop = FALSE])))
Sigma <- Reduce(`+`, lapply(seq_along(classes), function(i) {
  Xc <- sweep(X[y == classes[i], , drop = FALSE], 2, mu[i, ])
  crossprod(Xc)
})) / (n - K)
cat("priors:", round(pi_hat, 4), "\n\n")
```

```
priors: 0.6537 0.3463
```

```
print(round(t(`rownames<-`(mu, c("mu_0 (no CHD)", "mu_1 (CHD)"))), 3))
```

```
          mu_0 (no CHD) mu_1 (CHD)
sbp              135.460    143.738
tobacco            2.635      5.525
ldl                4.344      5.488
adiposity         23.969     28.120
famhist            1.318      1.600
typea             52.368     54.494
obesity           25.737     26.623
alcohol           15.931     19.145
age               38.854     50.294
```

```
cat("\nSigma dimension:", dim(Sigma), "\n")
```

```
Sigma dimension: 9 9
```

```
print(round(Sigma, 2))
```

```
             sbp tobacco   ldl adiposity famhist  typea obesity alcohol    age
sbp       405.43   14.59  4.58     49.16    0.34 -15.59   18.94   64.40  95.13
tobacco    14.59   19.24  0.76      7.54    0.02  -2.06    1.83   20.52  22.77
ldl         4.58    0.76  4.00      6.03    0.09   0.34    2.66   -2.53   6.48
adiposity  49.16    7.54  6.03     56.75    0.43  -5.31   22.71   16.12  60.51
famhist     0.34    0.02  0.09      0.43    0.23   0.08    0.18    0.77   1.00
typea     -15.59   -2.06  0.34     -5.31    0.08  95.57    2.64    7.96 -
20.28
obesity    18.94    1.83  2.66     22.71    0.18   2.64   17.62    4.69  15.70
alcohol    64.40   20.52 -2.53     16.12    0.77   7.96    4.69  598.28  27.89
age        95.13   22.77  6.48     60.51    1.00 -20.28   15.70   27.89 184.13
```

The priors are $\hat{\pi}_0 = 0.654$ (302/462) and $\hat{\pi}_1 = 0.346$ (160/462). The CHD group has a higher mean for every variable (for example `age` 50.3 vs 38.9, `tobacco` 5.5 vs 2.6, `ldl` 5.5 vs 4.3). Dimensions: $\mathbf{X}$ is 462×9, $\mathbf{y}$ has 462 entries, each μ_k has 9 entries, and Σ is 9×9.

b. Scores computed by hand, $\mathbf{x}_0^\top \Sigma^{-1} \mu_k - \frac{1}{2} \mu_k^\top \Sigma^{-1} \mu_k + \log \pi_k$, for all observations at once:

```
Sinv <- solve(Sigma)
const <- 0.5 * rowSums((mu %*% Sinv) * mu) - log(pi_hat)
scores <- sweep(X %*% Sinv %*% t(mu), 2, const)
cat("scores dimension:", dim(scores), "\n")
```

```
scores dimension: 462 2
```

```r
pred_hand <- classes[max.col(scores, ties.method = "first")]
cm_hand <- table(observed = y, predicted = pred_hand)
print(cm_hand)
```

```
        predicted
observed   0   1
       0 258  44
       1  73  87
```

```r
cat("training misclassification rate:", round(mean(pred_hand != y), 4), "\n")
```

```
training misclassification rate: 0.2532
```

The in-sample confusion matrix (rows observed, columns predicted) is

	pred 0	pred 1
obs 0	258	44
obs 1	73	87

so 117 of 462 are misclassified, a rate of 0.2532.

c. The same analysis with `MASS::lda`, using the empirical priors:

```r
fit_lda <- lda(X, grouping = y, prior = pi_hat)
pred_lda <- as.numeric(as.character(predict(fit_lda, X)$class))
print(table(observed = y, predicted = pred_lda))
```

```
        predicted
observed   0   1
       0 258  44
       1  73  87
```

```r
cat("identical labels to hand computation:", all(pred_lda == pred_hand), "\n")
```

```
identical labels to hand computation: TRUE
```

Yes, they match. The confusion matrix is identical and all 462 predicted labels agree with my hand calculation. That is what I expected: `MASS::lda` uses the same rule (class means, one pooled covariance matrix, and the empirical priors). It just computes it with a different matrix decomposition instead of inverting Σ directly.

Question 2: QDA and marginal screening

```
tr <- read.csv("data/zip-train.csv.gz", header = FALSE)
te <- read.csv("data/zip-test.csv.gz",  header = FALSE)
ytr <- tr[, 1]; Xtr <- as.matrix(tr[, -1])
yte <- te[, 1]; Xte <- as.matrix(te[, -1])
cat("train:", dim(Xtr), " test:", dim(Xte), "\n")
```

```
train: 7291 256  test: 2007 256
```

```
# pixels ranked by training variance
ord <- order(apply(Xtr, 2, var), decreasing = TRUE)

qda_zip <- function(m) {
  idx <- ord[1:m]
  fit <- qda(Xtr[, idx], grouping = factor(ytr))
  pred <- predict(fit, Xte[, idx])$class
  list(cm = table(observed = yte, predicted = pred),
       err = mean(as.character(pred) != as.character(yte)))
}
```

a. QDA with the 40 highest-variance pixels:

```
r40 <- qda_zip(40)
print(r40$cm)
```

```
        predicted
observed   0   1   2   3   4   5   6   7   8   9
       0 319   0  24   6   2   2   1   0   5   0
       1   0 238   5   0  11   0   7   0   3   0
       2   8   0 168   8   8   3   0   0   3   0
       3   5   0   4 128   0  14   0   0  15   0
       4   0   2  13   1 150   1   0   1   7  25
       5  12   0   3   5   1 131   0   0   6   2
       6   0   0   8   0   1   8 148   0   5   0
       7   1   0   6   1   6   0   0 124   2   7
       8   1   0  10  12   3   2   0   0 137   1
       9   0   0   1   2   7   1   1   4   9 152
```

```r
cat("test misclassification rate (40 predictors):", round(r40$err, 4), "\n")
```

```
test misclassification rate (40 predictors): 0.1555
```

The test misclassification rate is **0.1555** (312 of the 2,007 test images are wrong).

b. Yes, it works fairly well. QDA fits without any errors and classifies about 84% of the test digits correctly, far better than random guessing among ten classes (about 10%).

The reason it can be fitted is that 40 pixels is a manageable number of predictors. Each class's covariance matrix has $40 \cdot 41/2 = 820$ parameters, and every class has roughly 540 to 1,200 training images, which is more than 40, so each sample covariance matrix can be inverted. With all 256 pixels each class would need about $256 \cdot 257/2 \approx 32{,}900$ parameters, and many border pixels hardly vary at all, which is what made the full matrices singular in lecture. The condition numbers below show that the 40-pixel matrices are fine, although digit 1 is the closest to trouble (a thin stroke leaves several pixels nearly constant):

```r
round(sapply(0:9, function(k) kappa(cov(Xtr[ytr == k, ord[1:40]]), exact = TRUE)))
```

```
 [1]    1846 4465244     337     295     412     195     957    1807     119
[10]     636
```

I think it also works because the highest-variance pixels are most likely the ones near the center of the image, where strokes actually appear and differ between digits, so they carry more class information than the nearly blank edge pixels. QDA also lets each digit have its own pattern of variability, which suits handwriting.

It is far from perfect, though. The errors are concentrated in digits that look alike: 25 test fours were called nines and 24 zeros were called twos. The normal model is only a rough description of pixel values that are bounded between -1 and 1, and neighbouring pixels are highly correlated, so some of the 40 selected pixels are largely redundant.

c. With the 10 highest-variance pixels:

```r
r10 <- qda_zip(10)
print(r10$cm)
```

```
          predicted
observed   0   1   2   3   4   5   6   7   8   9
       0 298   0  11   1   1  16  28   4   0   0
       1   0 241  10   0   3   1   1   0   2   6
       2  19   2  95  14  14   3  22  21   5   3
```

```
  3   5   0   6 103   3  42   0   0   7   0
  4   1   5  14   0  95   2   4  15  17  47
  5  15   0   6  25   4  96   4   2   8   0
  6  41   0  15   4   0  18  78   7   5   2
  7   4   1   6   0   6   0   1 122   0   7
  8   1   2  11  36  12  11   4   1  74  14
  9   0   2   9   2  20   0   0  16  10 118
```

```
cat("test misclassification rate (10 predictors):", round(r10$err, 4), "\n")
```

```
test misclassification rate (10 predictors): 0.3423
```

The test error rises to **0.3423**, more than double the 40-pixel error (0.1555). The cause is not estimation trouble: with 10 predictors each covariance matrix has only 55 parameters and is estimated very well. The problem is bias: ten pixels just do not contain enough information to tell ten digits apart, and the confusion matrix shows many lookalike pairs mixed up (3 and 5, 4 and 9, 8 and 3, 2 and 6). Going from 10 to 40 predictors adds much more information than it adds estimation noise, so the larger model wins. Ranking pixels by variance alone also tends to choose pixels that sit next to each other and say similar things, so the first ten overlap a lot.

Question 3: Compare LDA and QDA

```
set.seed(432)
rhos <- c(0, 0.2, 0.4, 0.6, 0.8, 0.9)

gen <- function(n, rho) {
  S1 <- diag(10); S1[1, 2] <- S1[2, 1] <- rho
  S2 <- diag(10); S2[1, 2] <- S2[2, 1] <- -rho
  m1 <- c(-1, rep(0, 9)); m2 <- c(1, rep(0, 9))
  X <- rbind(mvrnorm(n, m1, S1), mvrnorm(n, m2, S2))
  colnames(X) <- paste0("x", 1:10)
  list(X = X, y = factor(rep(1:2, each = n)))
}

one_rep <- function(rho) {
  trn <- gen(100, rho); tst <- gen(1000, rho)
  l <- lda(trn$X, trn$y, prior = c(.5, .5))
  q <- qda(trn$X, trn$y, prior = c(.5, .5))
  c(LDA = mean(predict(l, tst$X)$class != tst$y),
```

```
    QDA = mean(predict(q, tst$X)$class != tst$y))
}
```

a. One dataset per ρ:

```
one <- t(sapply(rhos, one_rep)); rownames(one) <- paste0("rho = ", rhos)
round(one, 4)
```

```
             LDA    QDA
rho = 0   0.1615 0.2065
rho = 0.2 0.1630 0.1800
rho = 0.4 0.1660 0.2025
rho = 0.6 0.1765 0.1720
rho = 0.8 0.1640 0.1390
rho = 0.9 0.1690 0.0945
```

In this one draw QDA is worse at $\rho = 0, 0.2, 0.4$, the two methods are about even at $\rho = 0.6$, and QDA is clearly better at $\rho = 0.8$ and 0.9. A single draw is noisy, which is why part b repeats the experiment 50 times.

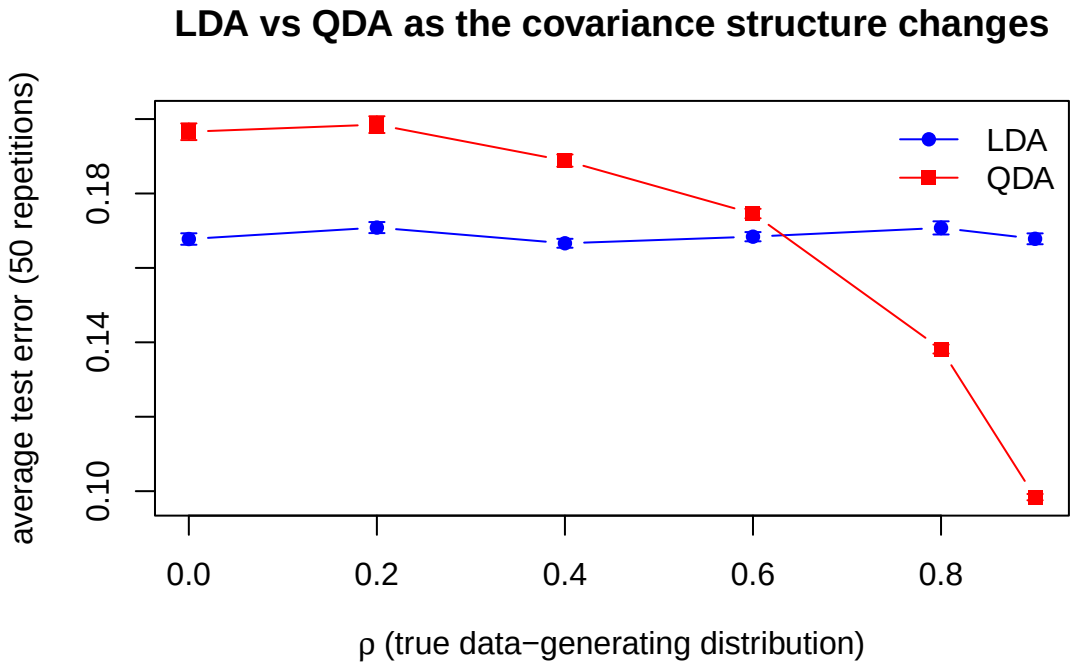
b. 50 repetitions per ρ:

```
res <- lapply(rhos, function(r) t(replicate(50, one_rep(r))))
avg <- t(sapply(res, colMeans)); rownames(avg) <- rhos
se  <- t(sapply(res, function(m) apply(m, 2, sd) / sqrt(50)))
print(round(cbind(avg, SE = se), 4))
```

```
       LDA    QDA    LDA    QDA
0   0.1678 0.1966 0.0015 0.0022
0.2 0.1708 0.1985 0.0015 0.0023
0.4 0.1666 0.1888 0.0012 0.0016
0.6 0.1684 0.1746 0.0012 0.0013
0.8 0.1707 0.1382 0.0018 0.0012
0.9 0.1678 0.0984 0.0015 0.0008
```

```
yl <- range(avg - se, avg + se)
plot(rhos, avg[, "LDA"], type = "b", pch = 16, col = "blue", ylim = yl,
     xlab = expression(rho ~ "(true data-generating distribution)"),
     ylab = "average test error (50 repetitions)",
     main = "LDA vs QDA as the covariance structure changes")
```

```r
lines(rhos, avg[, "QDA"], type = "b", pch = 15, col = "red")
arrows(rhos, avg[, "LDA"] - se[, 1], rhos, avg[, "LDA"] + se[, 1],
       angle = 90, code = 3, length = 0.04, col = "blue")
arrows(rhos, avg[, "QDA"] - se[, 2], rhos, avg[, "QDA"] + se[, 2],
       angle = 90, code = 3, length = 0.04, col = "red")
legend("topright", c("LDA", "QDA"), col = c("blue", "red"),
       pch = c(16, 15), lty = 1, bty = "n")
```



c. *Small* ρ. When $\rho = 0$ both classes have the same covariance (the identity), so LDA's assumption is exactly true. QDA is not wrong either, but it estimates a separate covariance matrix for each class (55 parameters each, 110 in total, versus 55 for LDA's single pooled matrix) from only 100 observations per class. That extra estimation noise costs accuracy: the average test error is 0.197 for QDA and 0.168 for LDA. The gap narrows as ρ grows, and at $\rho = 0.6$ LDA is still slightly ahead (0.168 vs 0.175).

Large ρ. The two classes now have opposite correlations ($+\rho$ and $-\rho$), so the best decision boundary is curved. LDA pools the two covariance matrices, and the opposite correlations cancel to roughly zero, so it cannot see this structure at all. Its error therefore stays about the same for every ρ (0.168 at $\rho = 0.9$). That is bias, and more data would not remove it. QDA fits each class's covariance separately, so it picks up the difference and its error keeps falling,

to 0.098 at $\rho = 0.9$. The curves cross between $\rho = 0.6$ and $\rho = 0.8$: at 0.6 LDA is still slightly better, and at 0.8 QDA is clearly better (0.138 vs 0.171). This is the bias-variance trade-off. LDA has low variance but is biased when the covariances differ; QDA has little bias but higher variance. QDA wins once LDA's bias costs more than QDA's extra variance.

The noise covariates. The eight noise variables say nothing about the class, but both methods still have to estimate parameters for them: a mean for each class, a variance, and covariances with every other variable. Each of those estimates is a bit off, and the errors add noise to the discriminant scores even though the true values carry no signal. QDA is hurt more because it has twice as many covariance parameters to estimate, which is part of why it does worse at small ρ. With fewer noise variables or more training data this penalty would shrink, and the crossover would move to a smaller ρ.

Question 4: Create and use an LDA/QDA analysis skill

a. The skill

The complete `SKILL.md` is appended at the end of this homework (see "Appendix: SKILL.md"). It is **3,583 bytes**, below the 5,000-byte limit.

b. Applying the skill to the `SAheart` data

Using the same response and predictor coding as Question 1, I followed the skill step by step: a stratified 30% test set reserved first (seed 432), the same stratified 10-fold split of the training data for every candidate, preprocessing (standardization and PCA) estimated within each fold, the one-standard-error rule for selection, and one single evaluation of the final model on the test set.

```
SEED <- 432; set.seed(SEED)
yf <- factor(y)

test_idx <- unlist(lapply(split(seq_along(yf), yf),
                          function(i) sample(i, round(0.3 * length(i)))))
Xtr4 <- X[-test_idx, ]; ytr4 <- yf[-test_idx]
Xte4 <- X[test_idx, ];  yte4 <- yf[test_idx]

fold_id <- integer(length(ytr4))
for (i in split(seq_along(ytr4), ytr4))
  fold_id[i] <- sample(rep(1:10, length.out = length(i)))

fit_predict <- function(spec, Xa, ya, Xb) {
  if (!is.na(spec$k)) {
```

```
    pc <- prcomp(Xa, center = TRUE, scale. = TRUE)
    Za <- pc$x[, 1:spec$k, drop = FALSE]
    Zb <- predict(pc, Xb)[, 1:spec$k, drop = FALSE]
  } else { Za <- Xa; Zb <- Xb }
  Pl <- predict(lda(Za, ya), Zb)$posterior
  Pq <- predict(qda(Za, ya), Zb)$posterior
  P <- switch(spec$kind, LDA = Pl, QDA = Pq, B = spec$w * Pq + (1 - spec$w) * Pl)
  colnames(P)[max.col(P, ties.method = "first")]
}

specs <- list(LDA = list(kind = "LDA", k = NA, w = NA),
              QDA = list(kind = "QDA", k = NA, w = NA))
for (k in 2:8) {
  specs[[paste0("PCA", k, "+LDA")]] <- list(kind = "LDA", k = k, w = NA)
  specs[[paste0("PCA", k, "+QDA")]] <- list(kind = "QDA", k = k, w = NA)
}
for (w in c(0.25, 0.5, 0.75))
  specs[[paste0("Blend w=", w)]] <- list(kind = "B", k = NA, w = w)

cv <- do.call(rbind, lapply(names(specs), function(nm) {
  e <- sapply(1:10, function(f) {
    a <- fold_id != f
    pr <- fit_predict(specs[[nm]], Xtr4[a, ], ytr4[a], Xtr4[!a, ])
    mean(pr != as.character(ytr4[!a]))
  })
  data.frame(model = nm, cv_error = mean(e), se = sd(e) / sqrt(10))
}))

simplicity <- function(nm) {
  if (nm == "LDA") 0
  else if (grepl("^PCA.*LDA$", nm)) 1
  else if (grepl("^Blend", nm)) 2
  else if (grepl("^PCA.*QDA$", nm)) 3
  else 4
}
imin <- which.min(cv$cv_error)
within <- cv[cv$cv_error <= cv$cv_error[imin] + cv$se[imin], ]
within$rank <- sapply(within$model, simplicity)
selected <- within$model[order(within$rank, within$cv_error)[1]]

pred4 <- fit_predict(specs[[selected]], Xtr4, ytr4, Xte4)
test_err <- mean(pred4 != as.character(yte4))
```

```r
test_se  <- sqrt(test_err * (1 - test_err) / length(yte4))
cm4 <- table(observed = yte4, predicted = pred4)
cat("selected:", selected, "| lowest CV:", cv$model[imin], "\n")
```

```
selected: LDA | lowest CV: PCA7+LDA
```

```r
cat(sprintf("test error: %.4f (SE %.4f)\n", test_err, test_se)); print(cm4)
```

```
test error: 0.3022 (SE 0.0389)
```

```
        predicted
observed  0  1
       0 76 15
       1 27 21
```

```r
cat("class counts (train):", table(ytr4), "\n")
```

```
class counts (train): 211 112
```

```r
cat("n train/test:", length(ytr4), length(yte4), "\n")
```

```
n train/test: 323 139
```

```r
Ztr <- scale(Xtr4)
ldaf <- lda(Ztr, ytr4)
cat("priors:", round(ldaf$prior, 3), "\n")
```

```
priors: 0.653 0.347
```

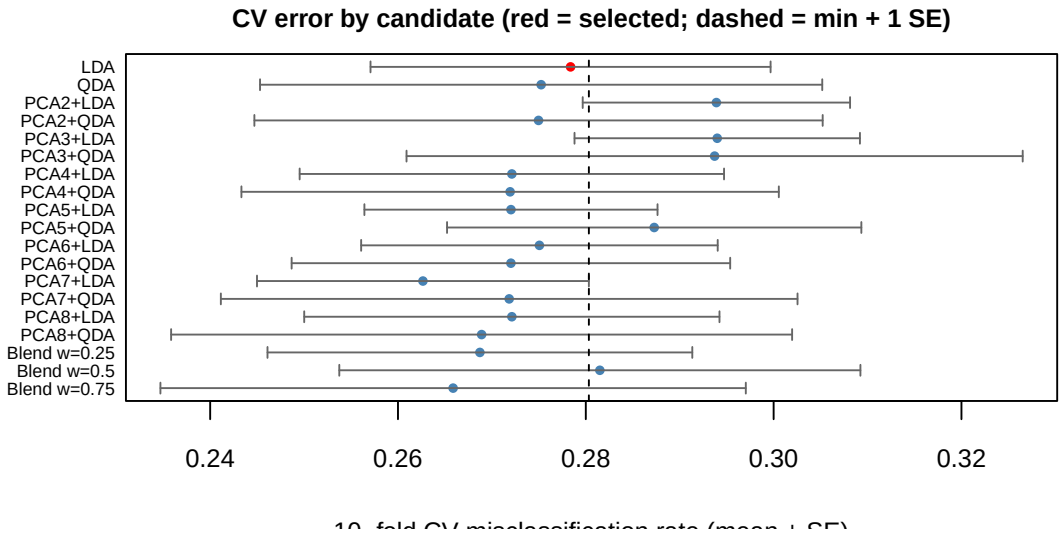
```r
coefs <- ldaf$scaling[, 1]
print(round(coefs[order(abs(coefs), decreasing = TRUE)], 2))
```

```
  tobacco       age       ldl     typea   famhist       sbp   obesity adiposity
     0.43      0.42      0.37      0.34      0.26      0.23     -0.21      0.15
  alcohol
    -0.01
```

c. One-page analysis report

One-page report: predicting coronary heart disease (`chd`) from nine risk factors with LDA/QDA (n = 462; seed 432)

Setup. Response `chd` (0 = 302, 1 = 160) with 9 numeric predictors (`famhist` coded 1/2). A stratified 30% test set (139 observations) was reserved first; the remaining 323 (211 / 112) were used for stratified 10-fold cross-validation (CV) with identical folds for all candidates. **Candidates** (19): LDA, QDA, LDA and QDA on the first $k = 2, \ldots, 8$ standardized principal components, and blends $wP_{QDA} + (1 - w)P_{LDA}$ with $w \in \{0.25, 0.5, 0.75\}$. Standardization and PCA were re-estimated within each training fold. Selection used the lowest CV misclassification rate, then the simplest model within one SE.



model	error	se	selected
LDA	0.278	0.021	yes
QDA	0.275	0.030	
PCA7+LDA	0.263	0.018	
Blend w=0.25	0.269	0.023	
Test error: LDA	0.302	0.039	test set

Fitted model. The selected model is plain LDA on all nine predictors, with training priors 0.653 / 0.347. On standardized predictors the largest discriminant coefficients are for tobacco, age, ldl, typea, so these do most of the work in separating the groups. `obesity` has a negative coefficient while `adiposity` is positive; these two variables are strongly correlated, so their individual signs should not be read too literally.

Findings. The lowest CV error was 0.263 (SE 0.018), for PCA7+LDA. Plain LDA scored 0.278, which is within one standard error of that, so the simplicity rule selects LDA. Plain

QDA scored 0.275, essentially the same as LDA, so the data do not show that separate class covariances help. Most differences among the 19 candidates are no bigger than their standard errors, so none of the PCA or blended versions is clearly better than plain LDA. The CV error of the selected model is about 7 points below the majority-class error in the training data (0.347), so the predictors do help. On the untouched test set the final model's error is **0.302 (SE 0.039)**, a bit higher than the CV estimate but well within one standard error of it. That is only about 4 points better than always predicting "no CHD" (0.345). The confusion matrix shows the model finds only 21 of the 48 CHD cases (sensitivity 44%), while specificity is 84%.

Limitations. The test set has only 139 observations, so the test error is imprecise and comes from one random split; a different seed could give a different split and a different near-tied winner. Differences among the top candidates are within noise. `famhist` is binary and several predictors are skewed, so the Gaussian assumptions behind LDA and QDA are only approximate. Finally, the 0.5 threshold and plain misclassification rate treat a missed CHD case and a false alarm as equally costly, which would not be true in practice.

How the skill was enabled for explicit invocation, and the prompt

- **File size:** `SKILL.md` is 3,583 bytes (limit 5,000).
- **Explicit invocation only:** The front matter contains `disable-model-invocation: true`, and the description says to use the skill only when the user invokes it by name.
- **Prompt used:**

  /lda-qda-report Use the attached SKILL.md. Dataset: `data/saheart.csv`. Response: `chd` (binary). Predictors: all nine other columns as numeric, with `famhist` coded 1 for Absent and 2 for Present. Seed: 432. Seek the best predictive performance within the LDA/QDA methods allowed by the skill, including ordinary LDA and QDA as comparisons. Produce the one-page report and show the code.

- **Review:** I checked the output against the skill's rules (test set reserved before modeling,shared folds, preprocessing re-fit inside each fold, one test evaluation). Nothing in the review depended on the test set, and I made no changes to the skill after the first run.

Appendix: SKILL.md

The file is 3,583 bytes. Long lines are wrapped below only so they fit on the page.

```
---
name: lda-qda-report
description: Guided LDA/QDA classification analysis with cross-validated model selection
and a one-page report. Use ONLY when the user explicitly invokes it by name (/lda-qda-
report). Never auto-load.
disable-model-invocation: true
---

# LDA/QDA analysis and one-page report

Inputs come from the user at invocation: a dataset, a categorical response, a list of
numeric predictors, and a random seed. Never assume names, sizes, or results. Use only
LDA, QDA, or combinations of them.

## 1. Setup
- Check the inputs: drop or report missing values, confirm the predictors are numeric,
  count observations per class, and note any class with too few observations to estimate
  a covariance matrix.
- Set and report the seed. Reserve a stratified test set (about 30%) **before any
  modeling**. Never use it for selection, tuning, or exploration.
- Build the cross-validation folds once (stratified, 10-fold, or fewer when classes are
```

```
  small) on the training data only. **Every candidate uses the same folds.**

## 2. Candidates
Always include ordinary LDA and ordinary QDA as baselines. Optionally add:
- LDA or QDA on the first k principal components, with k over a small grid up to the
  number of predictors.
- A blend of LDA and QDA class probabilities, `w*P_qda + (1-w)*P_lda`, with w on a small
  grid.
Choose settings by cross-validation only. Skip a candidate that cannot be fit (singular
covariance) and say so.

## 3. Cross-validation rules
- Misclassification rate is the only comparison metric.
- Any preprocessing (centering, scaling, PCA) is estimated **inside each fold from the
  training part of that fold only**, then applied to the held-out part.
- Report each candidate's mean CV error and its standard error across folds.
- Selection: take the lowest mean CV error, then choose the simplest candidate within
  one standard error of it. Simplicity order: LDA, PCA+LDA, blend, PCA+QDA, QDA.

## 4. Final evaluation
Refit the selected candidate, including preprocessing, on all training data. Evaluate it
**once** on the reserved test set. Report the test misclassification rate with a
binomial standard error, and the test confusion matrix (observed in rows, predicted in
columns). Do not revise the model after seeing the test result.

## 5. Report: at most ONE printed page
Use concise, readable prose, 10 pt or larger. Required parts, in this order:
1. **Title and setup** (2-3 lines): data size, response, number of predictors, class
   counts, split, folds, seed.
2. **Candidates and CV procedure** (2-4 lines).
3. **Figure**: dot plot or bar plot of mean CV error with standard-error bars for all
   candidates, with the selected one highlighted. Label axes, no clutter.
4. **Table**: candidates with mean CV error, standard error, and a flag for the selected
   model. Add a final row or second small table with test error and its standard error.
5. **Numerical fit results**: for the selected model, report class priors, class sizes,
   and tuning values (k, w), plus one fit summary such as class means or explained
   variance. Keep it short.
6. **Findings and limitations** (4-6 lines): which assumption (common vs separate
   covariance) the data favor, how large the differences are relative to their standard
   errors, and caveats such as small classes, non-normality, a single split, and
   differences between candidates that are within noise.
```

```
Report every number to a sensible precision. Never claim a difference matters when it is
within one standard error. Do not mention results from any other dataset. Output the
code used so the analysis is reproducible.
```